AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 5

The problem . . . 5

The solution . . . 5

How it works . . . 6

Why it matters . . . 6

Important boundaries . . . 6

Solution Overview — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 7

What this is . . . 7

How the technology works (the mental model) . . . 7

The standard deployment model . . . 8

Scope — what this solution covers . . . 8

Out of scope (handled elsewhere) . . . 9

Authentication and connections used . . . 9

Requirements — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 10

R1. Elevated deploy scope and role (Owner + User Access Administrator) . . . 10

R2. A hub/spoke network design decision (and address plan) . . . 10

R3. Private DNS strategy for the AI services . . . 11

R4. Allowed models, regions, and tagging standard (the guardrail inputs) . . . 11

R5. Resource-provider registration for the AI + networking services . . . 11

R6. Terraform remote state ready and encrypted (foundation-critical) . . . 12

R7. The IaC toolchain, pinned, with OIDC to Azure . . . 12

R8. A pilot (non-production) scope and change control . . . 13

Global Requirements — AI (category-level) 13

G1. Azure subscription with AI service access and rights . . . 13

G2. Entra identity, managed identity, and RBAC . . . 13

G3. Private networking capability . . . 13

G4. IaC + Python tooling and source control . . . 14

G5. CI/CD platform . . . 14

G6. Data source and classification (for data-grounded solutions) . . . 14

G7. Content-safety and responsible-AI posture . . . 14

G8. Cost controls and quotas . . . 14

Implementation — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 15

Phase 0 — Assess (read-only, changes nothing) . . . 16

Phase 1 — Configure (author the landing zone as code; nothing applied yet) . . . 17
Phase 2 — Pilot (apply to a non-production scope first, policies in audit) . . . 18
Phase 3 — Validate . . . 18
Phase 4 — Expand . . . 19
Rollback / safe-off . . . 19
Testing & Validation — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 20
Gate 1 — The hub/spoke network and peering exist . . . 20
Gate 2 — Private DNS zones exist and are linked to the VNets . . . 20
Gate 3 — AI service and Key Vault are private (public access disabled) . . . 20
Gate 4 — The AI hostname resolves to a private IP . . . 21
Gate 5 — The workload authenticates keyless (managed identity + RBAC) . . . 21
Gate 6 — The policy initiative is assigned and reporting (audit mode) . . . 21
Gate 7 — Content-safety default and responsible-AI posture are in place . . . 21
Gate 8 — Budget, alerts, and usage dashboard exist . . . 22
Gate 9 — Every change was plan/what-if-first and state is protected . . . 22
Troubleshooting — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 23
1. The AI (or Key Vault) hostname still resolves to a PUBLIC IP . . . 23
2. `terraform apply` — VNet peering fails . . . 23
3. Private endpoint stuck in "Pending" approval . . . 24
4. A policy is blocking legitimate work after you enforced it . . . 24
5. A guardrail policy definition "doesn't exist" / can't be assigned . . . 24
6. The pipeline can't authenticate to Azure (OIDC) . . . 24
7. `terraform init` fails on the backend / state is local . . . 25
8. The workload gets 401/403 calling the AI service (keyless) . . . 25
9. Budget exists but no alerts arrive . . . 25
10. GCC / GCC-High: a service, policy, or model isn't available . . . 25
11. Unexpected foundation cost . . . 26
Best Practices — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 26
Treat the landing zone as code, always . . . 26
Private by default — close the internet path . . . 26
Guardrails: audit first, then enforce — and keep the exits open . . . 27
Keyless identity over keys, every time . . . 27
Safety and cost are platform defaults, not afterthoughts . . . 27
Mind the cost of the foundation itself . . . 27
Where to go next . . . 28

Deliverables Summary — AI Landing Zone & Governance (Private Networking, Content Safety, Cost) 28
Documentation 28
Infrastructure-as-Code + helpers (5) 29
Named objects (verbatim — used across docs, IaC, and tests) 31
What this solution deliberately does NOT do 31
How it is delivered 32
Appendix A — PowerShell Scripts 33

Executive Summary — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

The problem

Most organizations adopt AI the same way: a team spins up an **Azure OpenAI** or **Azure AI Foundry** resource, wires it to some data, ships something useful — and then the next team does it all again, differently. Within a quarter you have a dozen AI resources, each having re-solved networking, identity, safety, and cost **on its own, inconsistently and insecurely**:

• **Public endpoints everywhere.** AI services reachable over the internet are a data-exfiltration surface — exactly the wrong default for services that touch your most sensitive data.

• **No guardrails.** Nothing *stops* a team from deploying an unapproved model, in an unapproved region, with public access on and content safety off.

• **Keys on laptops.** Service-to-service auth done with API keys that get copied into code and config, instead of managed identity and RBAC.

• **Bill shock.** Token-metered services with no budgets, no quotas, and no usage visibility until the invoice arrives.

Every one of these is a foundation problem, and foundation problems don't get better by adding more workloads on top of them.

The solution

This solution stands up a **governed AI landing zone** — the base every AI workload deploys **into**, so the networking, identity, safety, and cost decisions are made **once, correctly**, and inherited. It delivers:

• **A hub/spoke network with private endpoints for AI services** — Azure OpenAI, AI Search, Key Vault, and storage reached over **private endpoints and private DNS**, with **public network access disabled** — defined as **Infrastructure-as-Code**.

• **Azure Policy guardrails** — an initiative that enforces **allowed regions**, **allowed models**, **private-only** AI services, **content-safety-on**, and **required tagging** — deployed **audit/simulate-first**, then enforced.

• **An identity and RBAC baseline plus Key Vault** — **managed identity** and least-privilege data-plane roles for keyless auth, with Key Vault for any unavoidable secret.

• **Content-safety and responsible-AI defaults at the platform** — safety is a platform default, not a per-team afterthought.

• **Cost management** — **budgets, alerts, per-service quotas**, and a **usage dashboard**, so spend is visible and bounded before broad rollout.

How it works

We follow the same repeatable model every solution in this program uses — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. We first discover the current subscription and network state, then author the landing zone (network, private endpoints, Key Vault, RBAC, policy, cost controls) **as Terraform run through a plan/what-if-first pipeline**, apply it to a **non-production scope first** with the policy guardrails in **audit/DoNotEnforce** mode, validate that AI services are private, keyless, and guardrailed and that budgets fire, and only then **enforce** the policies and **expand** the landing zone across the estate. Every change is `terraform plan` **/** `what-if` **before apply**, in version control, approval-gated — never click-ops in the portal, never long-lived keys on a laptop.

The solution ships as **self-guided, click-by-click documentation plus real Infrastructure-as-Code** — a Terraform baseline (`main.tf`), a plan-first Azure Pipelines YAML pipeline (`azure-pipelines.yml`), and Az/PowerShell helpers that **discover** the current state, **validate** the guardrails, and **report** posture and cost — all **read-only or plan/what-if-first**, with **no secrets in code**.

Why it matters

- **Private by default** — AI services are unreachable from the public internet; access is over private endpoints inside your network.
- **Guardrails that actually stop things** — Azure Policy *prevents* the unapproved model, region, or public endpoint, rather than detecting it after the fact.
- **Keyless identity** — managed identity + RBAC instead of API keys that leak.
- **Safety and cost as platform defaults** — content safety on and budgets set before workloads land, not bolted on after an incident or an invoice.
- **A fraction of consulting cost** — the same disciplined method a cloud-platform team would use, delivered as a repeatable, tested runbook you reuse on every AI rollout.

Important boundaries

This solution builds the **governed foundation** — network, private endpoints, identity/RBAC, Key Vault, the policy guardrails, and cost controls. It **is not** an AI application, a RAG platform, an MLOps pipeline, or the content-safety product itself — securely deploying Azure OpenAI / AI Foundry, building a private-data RAG platform, standing up Azure ML pipelines, and configuring the content- safety and responsible-AI guardrails in depth are **separate catalog solutions** that this landing zone makes deployable. It **creates real, billable Azure resources** (private endpoints, Key Vault, Log Analytics, and any AI services placed in the zone have cost) — everything runs **plan/what-if first in a non-production scope before production**, and production apply is **the customer's change-approval decision**. Azure **Policy and feature availability, built-in policy definitions, model and region availability, GCC/GCC-High parity, and pricing all evolve and differ by cloud** — *verify current provider/module versions, policy/feature availability, and pricing before you apply*. The **acceptable-AI-use policy and content-acceptability thresholds** are the customer's legal / trust-and-safety / compliance decision — this guide provides the technical controls, not the policy.

Solution Overview — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

What this is

A complete, self-guided method for standing up a **governed AI landing zone on Azure** — a hub/spoke network with **private endpoints** for AI services, **Azure Policy** guardrails (allowed models/regions, private-only, content-safety-on, tagging), an **identity/RBAC baseline** with **Key Vault**, **content-safety and responsible-AI defaults** applied at the platform, and **cost management** (budgets, quotas, usage dashboards) — delivered as **real Infrastructure-as-Code** (Terraform + Az/PowerShell + a YAML pipeline) that runs **plan/what-if-first and approval-gated**.

It is the **foundation** layer for AI on Azure. Everything above it — a secure Azure OpenAI / AI Foundry deployment, a private-data RAG platform, MLOps pipelines — assumes this governed base exists and lands **into** it.

How the technology works (the mental model)

• **Landing zone.** An opinionated, pre-governed environment that workloads deploy **into** so they

inherit the network, identity, policy, and cost decisions instead of re-making them. An *AI* landing zone specializes this for AI services (Azure OpenAI, AI Foundry, AI Search, Content Safety) — which are unusually sensitive (they touch private data and can emit harmful output), so the defaults are **private, keyless, guardrailed, and budgeted**.

• **Hub/spoke network.** A **hub** virtual network holds shared platform services (private DNS,

connectivity, and later a firewall/gateway); one or more **spoke** virtual networks hold the AI workloads, **peered** to the hub. This centralizes DNS and connectivity and isolates workloads from each other. *Verify current hub/spoke reference topology for your connectivity model.*

• **Private endpoints + private DNS.** A **private endpoint** gives an Azure service (Azure OpenAI,

Key Vault, AI Search, storage) a **private IP inside your VNet**; **public network access is disabled** on the service. A **private DNS zone** (e.g. `privatelink.openai.azure.com`) makes the service's normal hostname resolve to that private IP. Result: the service is reachable **only** from inside your network — the internet path is closed.

• **Azure Policy + initiatives. Policy definitions** are rules evaluated against resources; an

initiative (policy set) groups them. Policies have **effects** — `Audit` (report only), `Deny` (block the create/update), `DeployIfNotExists`/`Modify` (remediate). Assigning an initiative at a **management-group or subscription scope** makes the rules apply to everything beneath it. Assigning in **DoNotEnforce (audit) mode first** lets you see impact before flipping to `Deny` — the simulate-before-enforce pattern. *Built-in policy definitions for AI services evolve — verify which definitions and effects exist in your cloud.*

• **Managed identity + RBAC.** A **managed identity** is an Azure-managed credential a workload uses to

authenticate to services **without keys**. **Azure RBAC** grants it least-privilege **data-plane** roles (e.g. *Cognitive Services OpenAI User*, *Key Vault Secrets User*). Keyless auth removes the single biggest AI

credential-leak vector.

• **Key Vault.** A managed secret/key store (itself reached via private endpoint) for any secret that genuinely can't be replaced by managed identity — accessed via RBAC, never embedded in code.

• **Content safety & responsible-AI defaults. Azure AI Content Safety** screens prompts and outputs for harmful content; enabling it (and responsible-AI defaults) **at the platform** means new AI workloads inherit safety rather than each team deciding. *Content-safety features and the responsible-AI toolset evolve and some are in preview — verify current availability. The acceptable- content thresholds and AI-use policy are the customer's trust-and-safety / legal decision.*

• **Cost management. Budgets** raise alerts as spend approaches thresholds; **quotas** (e.g. Azure OpenAI tokens-per-minute) cap consumption per deployment; a **usage dashboard/workbook** makes token and resource spend visible. AI bills on tokens/compute, so these are set **before** broad rollout.

The standard deployment model

Every solution in this program follows the same five stages:

1. **Assess.** Discover the subscription, registered resource providers, existing VNets/DNS, current policy assignments, Key Vault, and whether any AI services already exist with public access on (run `AILZ-Discovery.ps1`). Confirm deploy rights and remote state (category G1–G5).

1. **Configure.** Author the landing zone **as code** — hub/spoke VNets + peering, the private DNS zones, private endpoints, Key Vault, Log Analytics, the managed identity + RBAC, the policy initiative **in audit/DoNotEnforce mode**, and the budget — in `main.tf`, against remote state.

1. **Pilot.** Apply to a **non-production scope first** (a pilot subscription or resource group / management group), with policies **auditing, not denying**, using `terraform plan` **/** `what-if` **before every apply** (`azure-pipelines.yml` is plan-first and approval-gated).

1. **Validate.** Confirm AI services are **private** (no public path) and **keyless**, the policy initiative **reports** the expected compliance, the budget **alerts** fire, and content-safety defaults are in place (run `AILZ-Validate.ps1`, `AILZ-Report.ps1`).

1. **Expand.** Flip the policy guardrails from **audit to enforce (Deny)**, extend the assignment to the broader management-group scope, and institute the plan → review → apply loop for every future change.

Scope — what this solution covers

• **Hub/spoke network with private endpoints for AI services** — VNets, peering, private DNS zones, and private endpoints (with public access disabled), as code.

• **Azure Policy guardrails** — an initiative enforcing allowed regions, allowed models, private-only, content-safety-on, and required tagging, audit-first then enforced.

• **Identity & RBAC baseline + Key Vault** — managed identity, least-privilege data-plane roles, and a

private Key Vault for secrets.

• **Content-safety & responsible-AI defaults at the platform** — safety on by default for AI workloads that land in the zone.

• **Cost management** — budgets, alerts, per-service quotas, and a usage dashboard/workbook.

Out of scope (handled elsewhere)

• **A secure Azure OpenAI / AI Foundry deployment** — deploying and hardening the AI service itself is

the companion *Azure AI Foundry / OpenAI Secure Enterprise Deployment* solution (it lands **into** this zone).

• **A private-data RAG platform** — Azure AI Search over your permission-trimmed data is the companion

Enterprise RAG Platform solution.

• **MLOps** — Azure ML pipelines, registry, and monitoring are the companion *MLOps* solution.

• **Content-safety / responsible-AI in depth** — the full guardrail configuration, jailbreak/prompt-shield tuning, and RAI review workflow are the companion *Azure AI Content Safety & Responsible-AI Guardrails* solution; this landing zone sets the **defaults** it builds on.

• **The governance decisions** — which models and regions are allowed, the tagging standard, the budget thresholds, and every production apply approval are **the customer's cloud-governance / FinOps / trust-and-safety determinations**; this solution provides the technical guardrails and the code, and the **acceptable-AI-use policy is the customer's legal decision**.

Authentication and connections used

• **Azure Resource Manager** at a **subscription** (pilot) and later **management-group** scope, with **Owner or Contributor + User Access Administrator** (RBAC role assignments and policy assignment with remediation identities require the elevated role). *Verify the exact roles for policy remediation in your tenant.*

• **Terraform** (pinned `azurerm`/`azapi` provider — *verify current versions*) against **remote state **(an Azure Storage backend, category G4–G5), authenticating via a** workload-identity- federated (OIDC) pipeline identity **—** never a client secret on a laptop** (category G2, G5).

• **Az PowerShell** (`Az.Accounts`, `Az.Network`, `Az.Resources`, `Az.KeyVault`, `Az.PolicyInsights`, `Az.CostManagement` — *verify current module versions*) for the read-only discovery, validation, and reporting helpers, signed in with `Connect-AzAccount` (managed identity in CI).

• Everything is **plan/what-if-first**: `terraform plan` (or ARM `what-if`) precedes every apply, run through the pipeline with an approval gate. **GCC / GCC-High** use different clouds/endpoints and may lack some AI services or policies — *verify cloud-specific availability, endpoints, and pricing*.

Requirements — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

These are the **solution-specific** prerequisites for building the governed AI landing zone. The **category baseline** — an Azure subscription with AI-service access and Contributor/Owner rights, Entra identity + managed identity + RBAC, private-networking capability, the IaC + Python toolchain and Git, a CI/CD platform with OIDC federation, a data source + classification position, a content- safety / responsible-AI posture, and cost controls/quotas — is covered once in `global-requirements-ai.md` (**G1–G8**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. Elevated deploy scope and role (Owner + User Access Administrator)

• **Description:** The deployment identity has **Owner** (or **Contributor + User Access

Administrator**) at the target** subscription **(pilot) and, for the Expand stage, the target** management group**, plus** Resource Policy Contributor** for policy assignments.

• **Why required:** The landing zone creates **RBAC role assignments** and **policy assignments with

remediation identities**, both of which require rights beyond plain Contributor.

• **How to verify:** `Get-AzRoleAssignment -SignInName <you>` shows Owner/UAA at the scope; or run

`AILZ-Discovery.ps1`, which reports your effective role at the target scope. *Verify the exact roles required for policy remediation in your tenant.*

• **How to implement if missing:** Have an existing Owner grant the role at the subscription (pilot)

and management group (expand). Never use a personal admin account for the pipeline — use a workload-identity-federated service principal (G5).

R2. A hub/spoke network design decision (and address plan)

• **Description:** A short, written decision on the **hub/spoke topology** and a **non-overlapping IP

address plan** — the hub VNet CIDR, the AI spoke CIDR, and the subnet ranges for private endpoints and the AI workload.

• **Why required:** Private endpoints consume IPs from a subnet; peering requires non-overlapping

address space; DNS is centralized in the hub. Getting the address plan wrong is expensive to unwind after workloads land.

• **How to verify:** The CIDRs are documented, non-overlapping with existing/peered networks, and

large enough for growth. `AILZ-Discovery.ps1` lists existing VNets and their address spaces to check for overlap.

• **How to implement if missing:** Agree the topology and address plan with networking; set the CIDRs

in `main.tf` variables. If you already run a platform hub (e.g. an enterprise-scale landing zone), peer the AI spoke to it rather than creating a new hub. *Verify alignment with your existing connectivity model.*

R3. Private DNS strategy for the AI services

- **Description:** A decision on **where the private DNS zones live** (in this hub, or an existing

central private-DNS subscription) for each AI service you will make private — Azure OpenAI (`privatelink.openai.azure.com`), Cognitive/Content-Safety (`privatelink.cognitiveservices.azure.com`), AI Search (`privatelink.search.windows.net`), Key Vault (`privatelink.vaultcore.azure.net`), and blob storage (`privatelink.blob.core.windows.net`).

- **Why required:** Private endpoints only work if the service hostname resolves to the private IP;

that requires the correct private DNS zone **linked to the VNet**. Duplicating zones that already exist centrally causes resolution conflicts.

- **How to verify:** Confirm whether central private DNS zones already exist. `AILZ-Discovery.ps1`

lists private DNS zones in scope.

- **How to implement if missing:** Create the zones in the hub (this package does), **or** link the AI

spoke/hub to your existing central zones and disable zone creation in `main.tf`. *Verify the current privatelink zone names for each service — they occasionally change.*

R4. Allowed models, regions, and tagging standard (the guardrail inputs)

- **Description:** The written inputs the policy initiative enforces: the **allowed Azure regions**,

the **allowed AI models / model versions**, and the **required tags** (e.g. owner, cost-center, environment, data-classification).

- **Why required:** The `Policy-AllowedRegions-AILZ`, `Policy-AllowedModels-AILZ`, and

`Policy-RequireTags-AILZ` assignments **enforce the lists you set** — they do not choose them. The region list must match data-residency needs; the model list gates which models teams may deploy.

- **How to verify:** The lists are documented and owned by cloud governance. `AILZ-Report.ps1` shows

which values the assignments are currently enforcing.

- **How to implement if missing:** Facilitate a short session to agree the lists and set them in the

`main.tf` variables. *Which regions/models are allowed and which tags are mandatory are governance and data-residency decisions — the solution enforces what you set, it does not decide for you. Model and region availability differ by cloud — verify current availability.*

R5. Resource-provider registration for the AI + networking services

- **Description:** The Azure **resource providers** for the services in the zone are **registered** on

the subscription — e.g. `Microsoft.CognitiveServices`, `Microsoft.Network`, `Microsoft.KeyVault`, `Microsoft.OperationalInsights`, `Microsoft.PolicyInsights`,

`Microsoft.CostManagement`, and (where used) `Microsoft.MachineLearningServices` / `Microsoft.Search`.

• **Why required:** `terraform apply` / `what-if` fails if a required provider isn't registered on the subscription.

• **How to verify:** `Get-AzResourceProvider -ListAvailable | Where-Object RegistrationState -eq 'Registered'; or run AILZ-Discovery.ps1`, which flags any unregistered provider the zone needs.

• **How to implement if missing:** `Register-AzResourceProvider -ProviderNamespace <ns>` for each missing provider (registration can take a few minutes). *Azure OpenAI / some AI services are access-gated — verify your subscription has been granted access and the region is available.*

R6. Terraform remote state ready and encrypted (foundation-critical)

• **Description:** The **remote state backend** — an **Azure Storage account + container**, encrypted, versioned (blob versioning), with access via RBAC/OIDC — from category **G4–G5**, provisioned **before** the landing-zone apply.

• **Why required:** The landing zone is foundational; losing or corrupting its state is a serious incident. Remote, locked, versioned, encrypted state is mandatory here, not optional.

• **How to verify:** The state storage account/container exist, are encrypted and versioned, and the `backend "azurerm"` block in `main.tf` points at them. `terraform init` succeeds.

• **How to implement if missing:** Provision the Storage account + container first (a small bootstrap), enable versioning, configure the backend, and `terraform init`. Never run the landing zone on local state.

R7. The IaC toolchain, pinned, with OIDC to Azure

• **Description: Terraform CLI** (pinned), the **azurerm/azapi providers** (pinned), **Az PowerShell **modules, and** Git**, on the workstation/CI, authenticating to Azure via** workload-identity federation (OIDC)** — never a stored client secret (category G2, G5).

• **Why required:** Version drift breaks reproducibility of a foundation everything else depends on; OIDC removes long-lived secrets from the pipeline.

• **How to verify:** `terraform version`, `az version` / `Get-Module Az -ListAvailable`, and `git --version` return the pinned/supported builds; the pipeline's Azure service connection uses **workload identity federation** (no secret). *Verify current provider/module versions — the Azure providers and Az modules move fast.*

• **How to implement if missing:** Install and **pin** the toolchain (`required_version`, `required_providers`), and configure the pipeline's Azure service connection for OIDC. For **GCC / GCC-High**, set the correct Azure environment/endpoints — *verify cloud-specific configuration and service availability*.

R8. A pilot (non-production) scope and change control

• **Description:** A **non-production** target for the pilot — a dedicated pilot subscription, resource group, or management group — and a **change-approval** process for promoting to production.

• **Why required:** The whole method is **non-prod-first with policies in audit mode**; you need a safe scope to apply into and observe before enforcing guardrails and expanding.

• **How to verify:** A non-prod scope exists and the pipeline's approval gate (environment check) is configured. `AILZ-Discovery.ps1` confirms the target scope.

• **How to implement if missing:** Designate a pilot scope and configure the Azure Pipelines **Environment approval** so `apply` cannot run without a human approver. Production apply is the customer's change-approval decision.

Global Requirements — AI (category-level)

These requirements apply to every solution in the AI category (building and operating custom AI on Azure — Azure OpenAI / AI Foundry, RAG, Azure ML / MLOps, AI landing zones, content safety). Published once at the category level so listings stay small. Confirm before deploying any AI solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Custom AI touches sensitive data and can emit harmful or ungrounded output. Deploy private-by-default (no public endpoints, keyless auth), ground and permission-trim on your own data, put safety guardrails in front of generation, and pilot in non-production before production — content-acceptability and AI-use-policy decisions belong to the customer's legal / trust-and-safety / compliance functions, not this guide.*

G1. Azure subscription with AI service access and rights

Description: An Azure subscription with access to the required AI services (Azure OpenAI / AI Foundry, Azure AI Search, Azure Machine Learning, Azure AI Content Safety) and Contributor/Owner at the deployment scope. Why: These solutions create and configure real, billable Azure AI resources. Verify: `az account show` succeeds; the AI resource providers are registered and the region is available. *Azure OpenAI/model access and regions vary — verify current availability in your subscription.* Implement: Obtain the subscription/role; request access to the gated AI services where required; pick a supported region.

G2. Entra identity, managed identity, and RBAC

Description: Entra ID identities plus managed identities for keyless service-to-service auth. Why: Keys leak; AI services should authenticate via managed identity + RBAC (least privilege). Verify: Managed identities can be assigned and the needed data-plane roles exist (e.g. Cognitive Services OpenAI User, Search Index Data Reader). Implement: Enable managed identity on the compute/app; assign least-privilege data-plane roles; store any unavoidable secret in Key Vault.

G3. Private networking capability

Description: A VNet with the ability to create private endpoints and private DNS for the AI services. Why: Public endpoints are a data-exfiltration surface; enterprise AI is private-by-default. Verify: The VNet/subnets and Private DNS zones exist or can be created; public access can be disabled. Implement: Provision the VNet, subnets, and Private DNS; plan private endpoints per service (see the AI landing-zone solution).

G4. IaC + Python tooling and source control

Description: Terraform/Bicep, the Azure CLI, Python 3.x with the relevant Azure AI SDKs, and Git. Why: AI platforms are delivered as versioned code (infra + pipelines + app), so they are reproducible and reviewable. Verify: `az version`, `terraform version`/`az bicep version`, `python --version`, and `git --version` return supported builds. *Verify current SDK/provider versions — the AI SDKs move fast.* Implement: Install and pin the toolchain; store all config in Git.

G5. CI/CD platform

Description: A pipeline (GitHub Actions / Azure DevOps) with OIDC/workload-identity federation to Azure. Why: Infra, model, and app deployments must run through an auditable, approval-gated pipeline — not a laptop. Verify: The pipeline can authenticate to Azure via OIDC and run a plan/deploy step. Implement: Configure the service connection with workload-identity federation; scope its rights.

G6. Data source and classification (for data-grounded solutions)

Description: The private data to ground on (documents, databases) with a known permission/ACL model, and a sensitivity-classification position. Why: RAG must respect source permissions (permission-trimming) and must not expose sensitive data through AI. Verify: The data source, its access model, and applicable sensitivity labels are known. Run the solution's discovery script. Implement: Inventory the data and its ACLs; align sensitivity labeling (see the Purview solutions). *Data-classification determinations are the compliance team's decision.*

G7. Content-safety and responsible-AI posture

Description: An organizational position on acceptable AI content, safety thresholds, and responsible-AI review. Why: Generative AI can emit harmful/ungrounded output and be jailbroken; the organization must define guardrails and review. Verify: A responsible-AI / acceptable-use policy exists; content-safety is enabled where generation occurs. Implement: Adopt an RAI/acceptable-use policy and enable content safety (see the content-safety solution). *Acceptable-content and RAI determinations are the customer's legal / trust-and-safety decision.*

G8. Cost controls and quotas

Description: Budgets, alerts, and per-service quotas (e.g. Azure OpenAI TPM) appropriate to the workload. Why: AI services bill on tokens/compute; unmanaged usage causes bill shock. Verify: Budgets/alerts exist and quotas are set for the deployed services. Implement: Set budgets, cost alerts, and quotas/rate limits before broad rollout. *Spend limits are the customer's business decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

Implementation — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

This is the exact, ordered build. Every object you create is **named** with the convention `<purpose>-ailz-<scope/service>-<stage>` (resources) or `Policy-<Control>-AILZ` (policy assignments), and the **same name is reused** in later steps, testing, and the IaC. Everything is **plan/what-if-first**: `terraform plan` (or ARM `what-if`) precedes every apply, and you apply to a **non-production scope before production**, with the **policy guardrails in audit (DoNotEnforce) mode** until validated.

> ***Before you start:*** *confirm the elevated deploy role (R1), the address plan (R2), the private-DNS strategy (R3), the allowed regions/models/tags (R4), provider registration (R5), the encrypted* ***remote state*** *backend (R6), the pinned OIDC toolchain (R7), and a non-prod scope (R8). Sign in with a* ***workload-identity-federated*** *pipeline identity —* ***never a client secret on a laptop*** *(category G2, G5).*

Named objects created in this guide (verbatim — reused everywhere):

Object	Name	What it is
Resource group (hub)	`rg-ailz-hub-platform`	Hub network + private DNS + platform services
Resource group (AI spoke, pilot)	`rg-ailz-spoke-nonprod`	Pilot AI workload spoke
Hub VNet	`vnet-ailz-hub-platform`	Central connectivity + DNS
Spoke VNet	`vnet-ailz-spoke-nonprod`	AI workload network (pilot)
Subnet (private endpoints)	`snet-ailz-privateendpoints`	Holds the AI-service private endpoints
Subnet (AI workload)	`snet-ailz-aiworkload`	Compute/app that calls the AI services
VNet peering	`peer-ailz-hub-to-spoke` / `peer-ailz-spoke-to-hub`	Hub ■ spoke peering
Private DNS zone (OpenAI)	`privatelink.openai.azure.com`	Resolves Azure OpenAI to private IP
Private DNS zone (Cognitive/Content-Safety)	`privatelink.cognitiveservices.azure.com`	Resolves Cognitive Services / Content Safety
Private DNS zone (AI Search)	`privatelink.search.windows.net`	Resolves Azure AI Search
Private DNS zone (Key Vault)	`privatelink.vaultcore.azure.net`	Resolves Key Vault
Private DNS zone (blob)	`privatelink.blob.core.windows.net`	Resolves blob storage
Private endpoint (OpenAI)	`pep-ailz-openai-nonprod`	Private endpoint for the AI service

Object	Name	What it is
Private endpoint (Key Vault)	`pep-ailz-keyvault-nonprod`	Private endpoint for Key Vault
Key Vault	`kv-ailz-nonprod-<suffix>`	Secrets store (private, RBAC — globally unique name)
Log Analytics workspace	`log-ailz-platform`	Diagnostics + usage/workbook backing store
Managed identity (workload)	`id-ailz-aiworkload-nonprod`	Keyless identity for the AI workload
Policy initiative	`AILZ-AI-Guardrails-Initiative`	The grouped guardrail set
Policy assignment (regions)	`Policy-AllowedRegions-AILZ`	Restrict resource locations to the allow-list
Policy assignment (models)	`Policy-AllowedModels-AILZ`	Restrict Azure OpenAI model deployments
Policy assignment (private-only)	`Policy-PrivateOnly-AILZ`	Deny public network access on AI services
Policy assignment (content-safety)	`Policy-ContentSafetyOn-AILZ`	Audit/enforce content-safety posture
Policy assignment (tagging)	`Policy-RequireTags-AILZ`	Require the mandatory tags
Budget	`Budget-AILZ-Platform`	Cost budget + alert thresholds
Cost action group	`ag-ailz-cost-alerts`	Notifies owners on budget thresholds
Usage workbook	`Workbook-AILZ-AIUsage`	Token/resource usage dashboard
RBAC role assignment	`Cognitive Services OpenAI User` → `id-ailz-aiworkload-nonprod`	Keyless data-plane access

Phase 0 — Assess (read-only, changes nothing)

Step 0.1 — Confirm identity, scope, and providers

1. Sign in and confirm you are at the **pilot subscription** with the elevated role (R1):

```
Connect-AzAccount
Set-AzContext -Subscription "<pilot-subscription-id>"
```

1. Run the discovery helper:

```
./AILZ-Discovery.ps1 -SubscriptionId "<pilot-subscription-id>"
```

It reports: your **effective role** at the scope (expect Owner/UAA — R1), **registered vs missing resource providers** (R5), existing **VNets and address spaces** (to check overlap — R2), existing **private DNS zones** (R3), current **policy assignments**, any **Key Vault**, and — importantly — **any existing AI (Cognitive Services) resources with public network access ON**. **Read-only** — it calls only `Get`/`List` cmdlets.

Step 0.2 — Confirm remote state and toolchain

1. `terraform version`, `az version` (or `Get-Module Az -ListAvailable`), and `git --version` return

the pinned/supported builds (R7).

1. `terraform init` succeeds against the **remote state** backend (R6) — the Azure Storage account +

container, encrypted and versioned. Confirm the pipeline's Azure service connection uses **OIDC / workload identity federation**, not a stored secret.

Phase 1 — Configure (author the landing zone as code; nothing applied yet)

Step 1.1 — Set the variables

1. Open `main.tf` (and its `variables`/`terraform.tfvars`) and set:

• `location` and `allowed_regions` — your agreed region allow-list (R4).

• `allowed_models` — your agreed Azure OpenAI model/version allow-list (R4).

• `required_tags` — your mandatory tag keys (R4).

• `hub_address_space`, `spoke_address_space`, and the two subnet prefixes for

`snet-ailz-privateendpoints` and `snet-ailz-aiworkload` (R2) — non-overlapping.

• `create_private_dns_zones` — `true` to create the zones in the hub, or `false` to link to your

existing central zones (R3).

1. Confirm the object names match the table above (or your agreed convention).

Step 1.2 — Review the plan (no apply)

1. Generate and **read** the plan — this is the single most important review gate. In the pipeline

this is the `terraform plan` stage in `azure-pipelines.yml`; locally:

```
terraform plan -out ailz.plan
```

Confirm the plan will **create**: `rg-ailz-hub-platform` + `rg-ailz-spoke-nonprod`, the two VNets + subnets + `peer-ailz-*` peerings, the five `privatelink.*` DNS zones (if enabled) linked to the VNets, `pep-ailz-openai-nonprod` + `pep-ailz-keyvault-nonprod`, `kv-ailz-nonprod-<suffix>` (with **public access disabled**), `log-ailz-platform`, `id-ailz-aiworkload-nonprod` + its RBAC assignment, the `AILZ-AI-Guardrails-Initiative` and its five `Policy-*-AILZ` assignments **in** `DoNotEnforce` **(audit) mode**, and `Budget-AILZ-Platform` + `ag-ailz-cost-alerts` — and **nothing unexpected** (no deletions of existing infrastructure).

1. Commit the code and the plan output for review/approval (category G5, G8).

Phase 2 — Pilot (apply to a non-production scope first, policies in audit)

Step 2.1 — Apply the network, private endpoints, identity, and Key Vault

1. In the pipeline, run the **plan** → **approval** → **apply** flow (`azure-pipelines.yml`). The `apply`

stage runs **only after** the Environment approval gate is satisfied; it re-plans and applies `ailz.plan`. It targets the **pilot** scope (`rg-ailz-spoke-nonprod` / the pilot subscription).

1. Confirm `kv-ailz-nonprod-<suffix>` has **public network access = Disabled** and is reached via

`pep-ailz-keyvault-nonprod`; confirm `id-ailz-aiworkload-nonprod` holds the **Cognitive Services OpenAI User** role (keyless).

Step 2.2 — Assign the policy initiative in AUDIT (DoNotEnforce) mode

1. The `AILZ-AI-Guardrails-Initiative` and its five assignments (`Policy-AllowedRegions-AILZ`,

`Policy-AllowedModels-AILZ`, `Policy-PrivateOnly-AILZ`, `Policy-ContentSafetyOn-AILZ`, `Policy-RequireTags-AILZ`) are created with `enforcement_mode = "DoNotEnforce"` — they **report** non-compliance but do **not** block anything yet. This is deliberate: observe impact before enforcing.

1. Wait for the first compliance evaluation (it is not instant — *verify current evaluation timing*),

then review results with `AILZ-Report.ps1`. *Some built-in AI policy definitions may be preview or absent in your cloud — the initiative flags any that could not be assigned; verify availability.*

Step 2.3 — Place a test AI service and set content-safety + budget

1. Deploy **one pilot AI service** (e.g. an Azure OpenAI / Cognitive Services account) into

`rg-ailz-spoke-nonprod`, **public access disabled**, reached via `pep-ailz-openai-nonprod`, with the content-safety default in place. (Deploying and hardening the AI service in depth is the companion *Secure AI Foundry / OpenAI Deployment* solution — here you place one to prove the zone.)

1. Confirm `Budget-AILZ-Platform` is set with alert thresholds (e.g. 50/80/100%) wired to

`ag-ailz-cost-alerts`, and that `Workbook-AILZ-AIUsage` shows the pilot resource.

Phase 3 — Validate

1. Run the validation helper:

```
./AILZ-Validate.ps1 -SubscriptionId "<pilot-subscription-id>"
```

It checks the pass/fail gates in `05-testing.md`: VNets/peering/subnets exist, the private DNS zones are linked, the AI service and Key Vault have **public access disabled** and resolve to **private IPs**, the workload uses **managed identity (no keys)**, the `AILZ-AI-Guardrails- Initiative` is **assigned and reporting** (audit), the content-safety default is present, and `Budget-AILZ-Platform` exists with thresholds. It also runs a **private-resolution probe** (read-only) confirming the AI hostname resolves to a private IP.

1. Run the posture + cost report and review with governance:

```
./AILZ-Report.ps1 -SubscriptionId "<pilot-subscription-id>"
```

It summarizes private-endpoint coverage, policy compliance (what would be denied once enforced), budget status, and current AI spend. **Do not enforce or expand until the gates pass.**

Phase 4 — Expand

1. **Flip the guardrails from audit to enforce.** Once the compliance report shows the estate is clean (or the exceptions are understood and excepted), change the assignments' `enforcement_mode` from `DoNotEnforce` to `Default` **(enforced)** — `Policy-PrivateOnly-AILZ`, `Policy-AllowedRegions-AILZ`, `Policy-AllowedModels-AILZ`, and `Policy-RequireTags-AILZ` now **Deny** non-compliant creates/updates. Plan → review → apply as always.

1. **Extend the scope.** Move the initiative assignment from the pilot subscription up to the target **management group** so all AI workloads inherit the guardrails. Peer additional AI spokes to `vnet-ailz-hub-platform`.

1. **Institutionalize plan → review → apply.** Every future change to the network, policy, RBAC, or cost controls goes through `terraform plan` / `what-if`, review, and approval — never portal click-ops.

1. **Set the stage for the next layers.** With the zone governed, deploy the companion *Secure Azure OpenAI / AI Foundry*, *Enterprise RAG*, *MLOps*, and *Content Safety & Responsible-AI* solutions **into** it.

Rollback / safe-off

• **Because everything was plan/what-if-first and non-prod-first**, most missteps are caught in `plan` before they apply. Keep every apply small and reviewed.

• **A policy is denying something it shouldn't:** the guardrails ship in **audit mode** for exactly this reason. If you already enforced, set the assignment's `enforcement_mode` back to `DoNotEnforce` (or scope an **exemption** via an Azure Policy exemption for the specific resource), re-apply, and investigate. Never enforce a broad `Deny` estate-wide before validating in audit.

• **Terraform-managed objects:** `terraform destroy` (scoped, plan-first) removes what Terraform created — but **do not destroy the network, private DNS, or Key Vault casually**; other workloads may already depend on them. Treat destroy as a deliberate, reviewed action, and remember Key Vault soft-delete/purge-protection may retain the vault.

• **Break-glass:** keep an owner-level break-glass identity **exempt** from any deny policy that could lock out administration, and validate you can still manage the scope after enforcing.

• **Remote state** is your source of truth and rollback point — protect and version it (R6).

Testing & Validation — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

Validate the landing zone against explicit **pass/fail gates** before you **enforce** the policy guardrails or **expand** beyond the pilot scope. Run the helpers and walk the manual checks. Do not proceed to **Expand** until every gate passes. The automated gates are produced by `AILZ-Validate.ps1`; posture and cost evidence come from `AILZ-Report.ps1`; readiness from `AILZ-Discovery.ps1`; and every change went through the plan/what-if-first pipeline `azure-pipelines.yml` (with `main.tf` as the source of truth).

Each gate lists **What good looks like / How to test / If it fails**.

Gate 1 — The hub/spoke network and peering exist

- **What good looks like:** `vnet-ailz-hub-platform` and `vnet-ailz-spoke-nonprod` exist with the

agreed CIDRs, `snet-ailz-privateendpoints` and `snet-ailz-aiworkload` subnets are present, and `peer-ailz-hub-to-spoke` / `peer-ailz-spoke-to-hub` are **Connected**.

- **How to test:** `AILZ-Validate.ps1` lists the VNets, subnets, and peering state.
- **If it fails:** Re-check the address plan (R2) for overlap and re-apply the network resources

(Phase 2); peering fails if address spaces overlap.

Gate 2 — Private DNS zones exist and are linked to the VNets

- **What good looks like:** The five `privatelink.*` zones are present (or your central zones are

linked) and each is **VNet-linked** to the hub/spoke.

- **How to test:** `AILZ-Validate.ps1` lists the private DNS zones and their VNet links.
- **If it fails:** Create or link the zones (R3); an unlinked zone means the private hostname won't

resolve to the private IP. *Verify the current privatelink zone names.*

Gate 3 — AI service and Key Vault are private (public access disabled)

- **What good looks like:** The pilot AI (Cognitive Services / OpenAI) account and

`kv-ailz-nonprod-<suffix>` have **public network access = Disabled**, each with a private endpoint (`pep-ailz-openai-nonprod`, `pep-ailz-keyvault-nonprod`) in `snet-ailz-privateendpoints`.

- **How to test:** `AILZ-Validate.ps1` checks `publicNetworkAccess = Disabled` and lists the private

endpoints and their connection state.

- **If it fails:** Disable public access and add/repair the private endpoint (Phase 2). A service with

public access ON is the exact risk this zone removes.

Gate 4 — The AI hostname resolves to a private IP

- **What good looks like:** From inside the spoke, the AI service's hostname

(`<name>.openai.azure.com`) resolves to a **private IP** in `snet-ailz-privateendpoints`, not a public one.

- **How to test:** `AILZ-Validate.ps1` runs a read-only **private-resolution probe** (a DNS lookup /

private-endpoint DNS-record check) and reports the resolved address. *The probe is read-only and designed to be safe — verify it suits your environment; running it from outside the VNet will not see the private record.*

- **If it fails:** Confirm the private DNS zone is linked (Gate 2) and the private endpoint's DNS

A-record exists; fix the zone/VNet link.

Gate 5 — The workload authenticates keyless (managed identity + RBAC)

- **What good looks like:** `id-ailz-aiworkload-nonprod` holds the **Cognitive Services OpenAI User**

(and, for secrets, **Key Vault Secrets User**) role at the correct scope, and no API keys are used in workload config.

- **How to test:** `AILZ-Validate.ps1` lists the managed identity's role assignments and flags the

presence of a keyless path.

- **If it fails:** Assign the least-privilege data-plane role to the managed identity and remove any

key usage (category G2).

Gate 6 — The policy initiative is assigned and reporting (audit mode)

- **What good looks like:** `AILZ-AI-Guardrails-Initiative` is assigned at the pilot scope with its

five `Policy-*-AILZ` assignments in `DoNotEnforce` **(audit)** mode, and a **compliance evaluation** has produced results.

- **How to test:** `AILZ-Validate.ps1` confirms the assignment exists, its enforcement mode is

`DoNotEnforce`, and `AILZ-Report.ps1` shows per-policy compliance. *Any definition that is preview/ absent in your cloud is flagged as not-assigned — verify availability.*

- **If it fails:** Re-assign the initiative (Step 2.2); wait for evaluation; investigate any policy

that failed to assign (definition may be unavailable in your cloud).

Gate 7 — Content-safety default and responsible-AI posture are in place

- **What good looks like:** The content-safety posture is applied at the platform (the

`Policy-ContentSafetyOn-AILZ` assignment reports it, and the pilot AI service has the content-safety default), pending the deeper configuration in the companion Content-Safety solution.

• **How to test:** `AILZ-Validate.ps1` checks the content-safety policy assignment and the pilot service's content-safety signal. *Content-safety features / some responsible-AI tooling may be preview — verify current availability. Acceptable-content thresholds are the customer's trust-and- safety decision.*

• **If it fails:** Confirm the content-safety default is enabled and the policy assignment is present; defer threshold decisions to the customer's trust-and-safety function.

Gate 8 — Budget, alerts, and usage dashboard exist

• **What good looks like:** `Budget-AILZ-Platform` exists with alert thresholds wired to `ag-ailz-cost-alerts`, and `Workbook-AILZ-AIUsage` shows the pilot AI spend.

• **How to test:** `AILZ-Validate.ps1` confirms the budget and thresholds; `AILZ-Report.ps1` shows current spend vs budget and the usage workbook.

• **If it fails:** Create the budget + action group and confirm the workbook's Log Analytics source (`log-ailz-platform`). *Spend thresholds are the customer's FinOps decision.*

Gate 9 — Every change was plan/what-if-first and state is protected

• **What good looks like:** No portal click-ops changed the foundation; each apply followed a reviewed `terraform plan` behind the pipeline's approval gate; remote state is encrypted, versioned, and locked.

• **How to test:** Confirm the pipeline's plan/approval/apply history and that `apply` required the Environment approval; confirm the state backend is an encrypted, versioned Azure Storage container.

• **If it fails:** Bring drift back under Terraform (`terraform plan` shows drift), stop out-of-band portal changes, and protect the state backend (R6).

Validation summary

Gate	Check	Pass condition
1	Hub/spoke + peering	VNets/subnets exist; peerings Connected
2	Private DNS linked	Five `privatelink.*` zones VNet-linked
3	AI + Key Vault private	`publicNetworkAccess=Disabled` + private endpoints
4	Private resolution	AI hostname resolves to a private IP
5	Keyless identity	Managed identity holds data-plane roles; no keys
6	Policy initiative reporting	Assigned in `DoNotEnforce`; compliance produced

Gate	Check	Pass condition
7	Content-safety default	Content-safety assignment + service default present
8	Budget + usage	`Budget-AILZ-Platform` + `Workbook-AILZ-AIUsage` exist
9	Plan-first + state protected	Reviewed plans; encrypted/locked remote state

All nine gates pass ⇒ proceed to **Expand** (flip policies to enforce, extend to the management group). Any fail ⇒ stay in the pilot scope, keep policies in audit, and remediate the gate first.

Troubleshooting — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

The common failure modes, each as **symptom** → **cause** → **fix**. Most issues are DNS/private-endpoint wiring, policy scope/mode, or OIDC/state — not a broken service.

1. The AI (or Key Vault) hostname still resolves to a PUBLIC IP

• **Symptom:** From inside the spoke, `<name>.openai.azure.com` (or the vault hostname) resolves to a

public address; `AILZ-Validate.ps1` Gate 4 is not PASS.

• **Cause:** The `privatelink.*` **private DNS zone isn't linked to the VNet**, or the private

endpoint's DNS A-record wasn't created — so the public CNAME wins.

• **Fix:** Confirm the correct zone (e.g. `privatelink.openai.azure.com`) is **VNet-linked** to both

hub and spoke (Gate 2), and the private endpoint has a `private_dns_zone_group`. If you use **central** DNS zones (R3), set `create_private_dns_zones = false` and link there instead. *Verify the current privatelink zone name for the service.* Run the probe **from inside the VNet** — from outside you'll always see the public record.

2. `terraform apply` — VNet peering fails

• **Symptom:** `peer-ailz-hub-to-spoke` / `peer-ailz-spoke-to-hub` won't reach **Connected**.

• **Cause: Overlapping address space** between the hub and spoke (or with an already-peered

network), or only one side of the peering was created.

• **Fix:** Re-check the address plan (R2) — hub and spoke CIDRs must not overlap each other or any

peered network; `AILZ-Discovery.ps1` lists existing VNet address spaces. Both peering directions must exist. Re-apply.

3. Private endpoint stuck in "Pending" approval

• **Symptom:** `pep-ailz-openai-nonprod` / `pep-ailz-keyvault-nonprod` shows connection state

Pending, and traffic doesn't flow.

• **Cause:** The target resource is in a **different subscription/tenant** or has **manual private-endpoint approval** enabled, so the connection needs an owner to approve it.

• **Fix:** Approve the private-endpoint connection on the target resource (or set `is_manual_connection = false` when you own both sides). Confirm the connection state is **Approved**.

4. A policy is blocking legitimate work after you enforced it

• **Symptom:** After flipping a guardrail to enforce, valid deployments are **denied** (wrong region, a model you meant to allow, a missing tag).

• **Cause:** The allow-lists (`Policy-AllowedRegions-AILZ`, `Policy-AllowedModels-AILZ`) or the required-tag set don't match reality, or you enforced before validating in audit.

• **Fix:** The guardrails ship in **audit (DoNotEnforce)** mode for exactly this reason. Set the assignment's `enforcement_mode` back to `DoNotEnforce`, correct the allow-lists (R4), review the compliance report (`AILZ-Report.ps1`), and re-enforce. For a one-off legitimate exception, create a scoped **Azure Policy exemption** rather than weakening the policy.

5. A guardrail policy definition "doesn't exist" / can't be assigned

• **Symptom:** `terraform apply` errors that a policy **definition ID** isn't found, or a guardrail (private-only / allowed-models / content-safety) didn't make it into the initiative.

• **Cause:** The **built-in definition is preview, renamed, or absent in your cloud** (definitions differ between Commercial and GCC/GCC-High and evolve over time).

• **Fix:** `Get-AzPolicyDefinition | Where-Object {$_.Properties.DisplayName -like '*<term>*'}` to find the current definition, put the **verified ID** into the `local.guardrail_definition_ids` map in `main.tf` (an empty value drops that reference so the initiative still assigns). If no built-in fits, author a **custom** definition. *Verify current AI/content-safety policy availability for your cloud.*

6. The pipeline can't authenticate to Azure (OIDC)

• **Symptom:** `terraform init`/`plan` fails with an auth error; the service connection can't get a token.

• **Cause:** The ARM service connection isn't configured for **workload identity federation**, the federated-credential subject doesn't match the pipeline, or the identity lacks rights at the scope (R1).

• **Fix:** Recreate the service connection using **workload identity federation (OIDC)**, confirm the

federated credential's subject matches the pipeline/environment, and grant the identity **Owner or Contributor + User Access Administrator** at the target scope. Never fall back to a stored client secret.

7. `terraform init` fails on the backend / state is local

- **Symptom:** `init` errors on the `azurerm` backend, or state is sitting on the laptop.
- **Cause:** The remote-state Storage account/container isn't provisioned or the `backend "azurerm"` block isn't filled in (R6).
- **Fix:** Bootstrap the **encrypted, versioned** Storage account + container first, fill in the backend block (with `use_oidc = true`), and `terraform init`. **Never** run the foundation on local state.

8. The workload gets 401/403 calling the AI service (keyless)

- **Symptom:** The app authenticates with managed identity but is **denied** by Azure OpenAI / the vault.
- **Cause:** The **data-plane RBAC role** isn't assigned (or not at the right scope), or RBAC propagation hasn't completed. Key-based auth may also be disabled (as intended) while the role is missing.
- **Fix:** Confirm `id-ailz-aiworkload-nonprod` holds **Cognitive Services OpenAI User** at the AI resource scope (and **Key Vault Secrets User** on the vault), then wait for propagation. Data-plane roles are distinct from control-plane Contributor. `AILZ-Validate.ps1` Gate 5 lists the identity's roles.

9. Budget exists but no alerts arrive

- **Symptom:** `Budget-AILZ-Platform` is set but owners never get notified as spend climbs.
- **Cause:** The **action group** (`ag-ailz-cost-alerts`) has no receivers (the `budget_alert_emails` variable was left empty), or the notification thresholds aren't wired to it.
- **Fix:** Set `budget_alert_emails` and confirm the budget's notifications reference `ag-ailz-cost-alerts`. *Cost data can lag — verify current billing latency; spend thresholds are the customer's FinOps decision.*

10. GCC / GCC-High: a service, policy, or model isn't available

- **Symptom:** A resource, policy definition, or model that works in Commercial fails or is missing in a government cloud.
- **Cause: Feature, policy, model, and region parity differs by cloud**; some AI services and content-safety features aren't yet in GCC/GCC-High.
- **Fix:** Sign in with the correct environment (`Connect-AzAccount -Environment

AzureUSGovernment`), set the matching endpoints in the provider/backend, and **verify availability** per service before you plan. Substitute a supported region/model where needed. *Verify cloud-specific availability, endpoints, and pricing.*

11. Unexpected foundation cost

- **Symptom:** Bills climb after the landing zone goes in, before any real workload.
- **Cause: Private endpoints**, **Log Analytics ingestion/retention**, Key Vault operations, and

any AI services placed in the zone all bill, and cost scales with traffic and workloads.

- **Fix:** Right-size Log Analytics retention (`log-ailz-platform`), review spend per resource group

with `AILZ-Report.ps1`, and set quotas (e.g. Azure OpenAI tokens-per-minute). *Verify current Azure pricing for your cloud — pricing changes. Cost trade-offs are a customer FinOps decision.*

Best Practices — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

Keep-it-clean guidance for a governed AI landing zone that stays private, safe, budgeted, and reproducible over time.

Treat the landing zone as code, always

- **No portal click-ops on the foundation.** The network, private DNS, private endpoints, Key Vault,

RBAC, policy, and cost controls change through `terraform plan` / `what-if` → review → apply (`azure-pipelines.yml`), in version control. A foundation changed by hand is a foundation no one can reproduce or audit.

- **Plan/what-if-first, non-prod-first.** Read every plan before applying, and apply to the pilot

scope before production. The plan is your safety net — use it.

- **Protect the state like production data.** Remote, encrypted, versioned, locked state (R6) is the

source of truth for the whole AI estate; losing it is an incident. Restrict who can touch the state backend.

Private by default — close the internet path

- **Public access OFF, private endpoints ON.** Every AI service, Key Vault, and storage account in the

zone has **public network access disabled** and is reached over a private endpoint. If a service can be reached from the internet, it is not in the landing zone's posture.

- **Get DNS right or nothing resolves.** The private endpoint is only useful if the service hostname

resolves to its private IP — keep the `privatelink.*` zones **linked to the VNets** and prefer your organization's **central** private DNS if one exists, rather than duplicating zones. *Verify current privatelink zone names.*

- **One hub, many spokes.** Centralize DNS and connectivity in `vnet-ailz-hub-platform`; peer each new

AI workload spoke to it, rather than each team rebuilding networking.

Guardrails: audit first, then enforce — and keep the exits open

• **Simulate before you enforce.** Assign the `AILZ-AI-Guardrails-Initiative` in **`DoNotEnforce`

(audit) **mode, read the compliance report, understand the impact, then flip to** enforce (Deny)**. Never enforce a broad deny estate-wide before validating in audit.

• **Enforce what you decided, not more.** `Policy-AllowedRegions-AILZ`, `Policy-AllowedModels-AILZ`,

`Policy-PrivateOnly-AILZ`, and `Policy-RequireTags-AILZ` enforce the **lists you set** (R4) — the solution doesn't choose your regions, models, or tags.

• **Never lock out administration.** Keep an **owner-level break-glass identity exempt** from any deny

policy that could block management, and confirm you can still operate the scope after enforcing. Use scoped **Azure Policy exemptions** for legitimate exceptions rather than weakening the policy.

• **Fewer, clearer policies.** A handful of well-named guardrails beats a sprawl of overlapping

assignments — keep the initiative legible. *Built-in AI policy definitions evolve and some are preview — verify availability before relying on an effect.*

Keyless identity over keys, every time

• **Managed identity + RBAC, not API keys.** Give the workload (`id-ailz-aiworkload-nonprod`)

least-privilege data-plane roles (e.g. *Cognitive Services OpenAI User*, *Key Vault Secrets User*). Keys are the single biggest AI credential-leak vector — remove them.

• **Key Vault for the unavoidable secret only, and privately.** Reach Key Vault over its private

endpoint, grant access via RBAC, and keep purge-protection on. Don't put secrets in code, pipeline variables in plaintext, or app config.

Safety and cost are platform defaults, not afterthoughts

• **Content safety on at the platform.** New AI workloads should inherit content-safety defaults, not

each team deciding — the deeper tuning (prompt shields, jailbreak detection, RAI review) is the companion Content-Safety solution. *Acceptable-content thresholds and the AI-use policy are the customer's trust-and-safety / legal decision, not this guide's.*

• **Budget and quota before rollout.** Set `Budget-AILZ-Platform` thresholds and per-service quotas

(e.g. Azure OpenAI tokens-per-minute) **before** workloads scale. Watch `Workbook-AILZ-AIUsage`. *Spend limits are the customer's FinOps decision; verify current AI pricing — it changes.*

Mind the cost of the foundation itself

• **The zone is billable.** Private endpoints, Log Analytics ingestion, Key Vault operations, and any

AI services placed in the zone all cost money, and cost scales with workloads and traffic. Right-size Log Analytics retention and review spend per resource group. *Verify current Azure pricing for your cloud (Commercial vs GCC/GCC-High) before you scale.*

Where to go next

With the governed AI landing zone in place, the natural next steps in this catalog are:

- **Secure Azure AI Foundry / OpenAI Deployment** — deploy and harden the AI service itself **into** this zone (private, keyless, content-safety-on) now that the foundation exists.
- **Enterprise RAG Platform (Azure AI Search, private data)** — ground AI on your permission-trimmed data using the private network and identity baseline this zone provides.
- **Azure AI Content Safety & Responsible-AI Guardrails** — take the platform content-safety default and configure it in depth (prompt shields, jailbreak detection, RAI review workflow).
- **MLOps (Azure Machine Learning pipelines, registry, monitoring)** — operationalize custom models on the same governed base.
- **Azure Security Baseline & Governance** — apply the same governed-foundation discipline to the broader Azure estate for a consistent posture.

Deliverables Summary — AI Landing Zone & Governance (Private Networking, Content Safety, Cost)

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Executive Summary (`executive-summary.md`)	One-page business framing: the AI-sprawl problem (public endpoints, no guardrails, keys, bill shock), the governed AI landing zone solution, and the boundary (foundation only; secure AI deployment / RAG / MLOps / content-safety-in-depth are separate; plan-first, production apply and AI-use policy are the customer's decisions).
Solution Overview (`01-solution-overview.md`)	What it is, how the technology works (landing zone, hub/spoke, private endpoints + private DNS, Azure Policy initiatives + audit-first, managed identity + RBAC, Key Vault, content-safety defaults, cost management), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used.

Document	Purpose
Requirements (`02-requirements.md`)	Solution-specific prerequisites (elevated deploy role, hub/spoke + address plan, private-DNS strategy, allowed regions/models/tags, provider registration, encrypted remote state, the pinned OIDC toolchain, a non-prod scope) — layered on the shared `global-requirements-ai.md` (G1–G8).
Implementation (`03-implementation.md`)	Exact, ordered build: discover state, author the zone as code, plan/what-if-first apply of network/private-endpoints/identity/Key Vault/policy(audit)/budget to a non-prod scope, place a pilot AI service, validate private+keyless+guardrailed, then flip policies to enforce and expand to the management group — every object named, with rollback.
Testing & Validation (`05-testing.md`)	Nine pass/fail gates from hub/spoke+peering, private DNS, public-access-disabled, private-resolution, keyless identity, policy-reporting-in-audit, content-safety default, budget+usage, through plan-first-with-protected-state, referencing the helpers by name.
Troubleshooting (`06-troubleshooting.md`)	The common failure modes (private hostname resolving public, peering fails on overlap, private-endpoint approval pending, policy blocking legitimate work, definition unavailable/preview, OIDC/state failures, keyless 401, budget silent, GCC/GCC-High gaps, foundation cost) as symptom → cause → fix.
Best Practices (`07-best-practices.md`)	Foundation-as-code, private-by-default, audit-then-enforce-and-keep-exits-open, keyless-over-keys, safety-and-cost-as-defaults, mind-the-cost — plus where to go next in the catalog.

Infrastructure-as-Code + helpers (5)

The core is **real Terraform**; the pipeline is **plan/what-if-first and approval-gated**; the helpers are **read-only**. Nothing uses secrets — everything authenticates via **workload-identity federation (OIDC)** / interactive `Connect-AzAccount` (never a client secret in code). *Verify current provider/ module versions, built-in policy-definition availability (some AI/content-safety definitions may be preview or absent), privatelink zone names, model/region availability, and pricing for your cloud (Commercial vs GCC/GCC-High).*

File	Type	What it does
`main.tf`	Terraform (plan-first)	Declares the AI landing zone as code: hub/spoke VNets + peering, the five `privatelink.*` DNS zones (linked), the OpenAI + Key Vault private endpoints, `kv-ailz-nonprod-<suffix>` (public access **disabled**), `log-ailz-platform`, `id-ailz-aiworkload-nonprod` + RBAC, the `AILZ-AI-Guardrails-Initiative` and its five `Policy-*-AILZ` assignments in **DoNotEnforce (audit)** mode, and `Budget-AILZ-Platform` + `ag-ailz-cost-alerts`. Applied only through the pipeline.
`azure-pipelines.yml`	YAML pipeline (plan-first, approval-gated)	The **only** sanctioned way to apply `main.tf`: fmt/validate → **plan** (changes nothing, publishes the plan artifact) → **Environment approval gate** → **apply** the reviewed plan (non-prod first, never on a PR) → **read-only validate**. Authenticates via OIDC.
`AILZ-Discovery.ps1`	Read-only (Az PowerShell)	Inventories current state: effective role, registered/missing providers, existing VNets + address spaces (overlap check), private DNS zones, policy assignments, Key Vaults, and any **AI service with public access ON**. Calls only Get/List.
`AILZ-Validate.ps1`	Read-only (Az PowerShell)	Runs the `05-testing.md` gates as PASS/FAIL/MANUAL: hub/spoke+peering, private DNS, public-access-disabled, a private-resolution DNS probe, keyless identity, policy assigned+audit, content-safety default, and budget — with a non-zero exit on hard failure.

File	Type	What it does
`AILZ-Report.ps1`	Read-only (Az PowerShell)	Posture + cost snapshot for governance/FinOps: private-endpoint coverage, guardrail compliance (what would be denied once enforced), budget vs actual spend, and AI spend by service.

Named objects (verbatim — used across docs, IaC, and tests)

- **Resource groups:** `rg-ailz-hub-platform`, `rg-ailz-spoke-nonprod`
- **Network:** `vnet-ailz-hub-platform`, `vnet-ailz-spoke-nonprod`, `snet-ailz-privateendpoints`,

`snet-ailz-aiworkload`, `peer-ailz-hub-to-spoke`, `peer-ailz-spoke-to-hub`

- **Private DNS zones:** `privatelink.openai.azure.com`, `privatelink.cognitiveservices.azure.com`,

`privatelink.search.windows.net`, `privatelink.vaultcore.azure.net`, `privatelink.blob.core.windows.net`

- **Private endpoints:** `pep-ailz-openai-nonprod`, `pep-ailz-keyvault-nonprod`
- **Platform:** `kv-ailz-nonprod-<suffix>` (Key Vault), `log-ailz-platform` (Log Analytics),

`id-ailz-aiworkload-nonprod` (managed identity), `Workbook-AILZ-AIUsage` (usage dashboard)

- **Policy:** `AILZ-AI-Guardrails-Initiative` with `Policy-AllowedRegions-AILZ`,

`Policy-AllowedModels-AILZ`, `Policy-PrivateOnly-AILZ`, `Policy-ContentSafetyOn-AILZ`, `Policy-RequireTags-AILZ`

- **Cost:** `Budget-AILZ-Platform`, `ag-ailz-cost-alerts`
- **RBAC:** `Cognitive Services OpenAI User` (and `Key Vault Secrets User`) → `id-ailz-aiworkload-nonprod`

What this solution deliberately does NOT do

- It does **not** deploy or harden the AI service itself (Azure OpenAI / AI Foundry), build a

private-data RAG platform, stand up MLOps pipelines, or configure content-safety / responsible-AI in depth — those are separate catalog solutions this landing zone makes deployable.

- It does **not** apply anything without a plan — every change is **plan/what-if-first, non-prod-first**,

guardrails start in **audit (DoNotEnforce)** mode, and production apply / policy enforcement require the customer's change approval.

- It does **not** decide **which models/regions are allowed, the tagging standard, or the budget

thresholds **— those are governance/FinOps determinations; the solution enforces what you set. The** acceptable-AI-use policy and content-acceptability thresholds are the customer's legal / trust-and- safety

decision.**

- It does **not** use client secrets or API keys — the pipeline uses **OIDC** and the workload uses **managed identity**.
- It does **not** invent version, feature, policy, or pricing facts — where these are preview or differ by cloud it says **verify current provider/module versions, policy/feature availability, and pricing** (Commercial vs GCC/GCC-High).

How it is delivered

This solution is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus the Terraform, YAML pipeline, and Az/PowerShell files, in a single ZIP (`AILZ-Complete-Solution.zip`) — yours to keep and reuse on every AI rollout.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AILZ-Discovery.ps1

Purpose: Discovers the starting reality before any change: your effective role at the target scope, registered vs missing resource providers, existing VNets and their address spaces (to check for overlap), existing private DNS zones, current policy assignments, any Key Vault, and — importantly — any existing AI (Cognitive Services / OpenAI) resources with PUBLIC network access still ON. Read-only: it calls only Get/List cmdlets.

How to run: Connect-AzAccount ./AILZ-Discovery.ps1 -SubscriptionId "<pilot-subscription-id>" Prereqs: Az PowerShell (Az.Accounts, Az.Resources, Az.Network, Az.KeyVault, Az.CognitiveServices, Az.PolicyInsights — VERIFY current module versions). For GCC/GCC-High, Connect-AzAccount -Environment AzureUSGovernment first.

AILZ-Report.ps1

Purpose: Produces a governance-facing snapshot: private-endpoint coverage across AI services / Key Vault, which guardrails the AILZ-AI-Guardrails-Initiative is enforcing and its current compliance (what WOULD be denied once enforced), budget status vs actual spend, and current AI spend by service. Read-only: Get/List + policy-state + cost queries only.

How to run: Connect-AzAccount ./AILZ-Report.ps1 -SubscriptionId "<pilot-subscription-id>"

AILZ-Validate.ps1

Purpose: Validates the deployed landing zone against the gates in 05-testing.md and prints each as PASS / FAIL / MANUAL: hub/spoke VNets + peering, private DNS zones linked, AI service and Key Vault public-access DISABLED with private endpoints, the AI hostname resolving to a PRIVATE IP (read-only probe), keyless managed identity + RBAC, the AILZ-AI-Guardrails-Initiative assigned and reporting in audit (DoNotEnforce) mode, content-safety default present, and Budget-AILZ-Platform with thresholds. Read-only: Get/List + a DNS lookup only.

How to run: Connect-AzAccount ./AILZ-Validate.ps1 -SubscriptionId "<pilot-subscription-id>" Optionally pass -AiAccountName to target a specific AI service for the private-resolution probe.

azure-pipelines.yml

Purpose: The ONLY sanctioned way to apply main.tf. Runs Terraform PLAN first (changes nothing), then requires a HUMAN APPROVAL (Environment check) before APPLY, and targets a NON-PRODUCTION scope first. Authenticates to Azure via WORKLOAD IDENTITY FEDERATION (OIDC) — no client secret. A read-only validate stage runs AILZ-Validate.ps1 after apply. HOW TO RUN - Create an Azure Resource Manager service connection using WORKLOAD IDENTITY FEDERATION (OIDC) named per $(azureServiceConnection). - Create an Environment named 'ailz-nonprod' with a required-approvals check (this is the approval gate that guards Apply). For production, add an 'ailz-prod' environment with its own approvers and run against prod vars. - Provide the remote-state backend config (R6) as pipeline variables/backend. EXPECTED RESULTS Plan artifact published for review -> manual approval -> apply -> validate. Apply cannot run without the Environment approval. Nothing applies on a PR. NOTES

White-label. Plan/what-if-first, non-prod-first, approval-gated. No secrets in this file. VERIFY task versions, provider versions, and cloud (Commercial vs GCC/GCC-High) endpoints before use.

How to run: - Create an Azure Resource Manager service connection using WORKLOAD IDENTITY FEDERATION (OIDC) named per $(azureServiceConnection). - Create an Environment named 'ailz-nonprod' with a required-approvals check (this is the approval gate that guards Apply). For production, add an 'ailz-prod' environment with its own approvers and run against prod vars. - Provide the remote-state backend config (R6) as pipeline variables/backend. EXPECTED RESULTS Plan artifact published for review -> manual approval -> apply -> validate. Apply cannot run without the Environment approval. Nothing applies on a PR. NOTES White-label. Plan/what-if-first, non-prod-first, approval-gated. No secrets in this file. VERIFY task versions, provider versions, and cloud (Commercial vs GCC/GCC-High) endpoints before use.

main.tf

Purpose: Declares the governed AI landing zone as code: a hub/spoke VNet with peering, private DNS zones and private endpoints for AI services (Azure OpenAI / Cognitive Services, Key Vault, AI Search, storage), a private Key Vault, a Log Analytics workspace, a keyless workload managed identity + RBAC, an Azure Policy initiative of AI guardrails (allowed regions/models, private-only, content-safety-on, tagging) assigned in AUDIT (DoNotEnforce) mode, and a cost budget + alert action group. HOW TO RUN (always plan/what-if-first, non-prod-first — see 03-implementation.md) Use the plan-first, approval-gated pipeline (azure-pipelines.yml): terraform plan -out ailz.plan # never changes anything terraform apply ailz.plan # runs ONLY after the approval gate EXPECTED RESULTS A plan that CREATES the two resource groups, hub+spoke VNets/subnets/peering, the five privatelink DNS zones (linked), the OpenAI + Key Vault private endpoints, kv-ailz-nonprod-<suffix> (public access DISABLED), log-ailz- platform, id-ailz-aiworkload-nonprod + its RBAC, the AILZ-AI-Guardrails- Initiative and its five Policy-*-AILZ assignments in DoNotEnforce, and Budget-AILZ-Platform + ag-ailz-cost-alerts. Nothing is applied by `plan`. NOTES White-label. Plan/what-if-first. No secrets in code — authenticate via the pipeline's workload-identity federation (OIDC), never a client secret. Policies ship in DoNotEnforce (audit) mode; flip to enforce only after validation (see 03-implementation.md Phase 4). VERIFY current azurerm/azapi provider versions, built-in policy definition IDs and availability (some AI/content-safety definitions may be preview or absent), privatelink zone names, model/region availability, and pricing for your cloud (Commercial vs GCC/GCC-High) before applying.